Why do we need a model of computation?

Before designing algorithms, we need a standard way to measure:

- How much time an algorithm takes
- How many operations it performs
- How memory access works
- It helps us analyze sequential algorithms

In sequential algorithms, we use the Random Access Machine (RAM) model.

The **RAM (Random Access Machine) model** is a theoretical model of a sequential computer where the processor can access any memory location in constant time, and executes instructions one at a time.

Key assumptions of RAM model:

The RAM model assumes:
Single processor: Only one CPU executes instructions.
Random access memory: Any memory cell can be accessed in O(1) time.
Unit cost operations: Basic operations take constant time such as addition, subtraction, load/store.

Limitations of RAM model:

RAM model does not represent parallel systems because of the presence of only one processor, no simultaneous execution, no parallel memory access conflicts.

So for parallel algorithms, we need a new model which is PRAM model (Parallel RAM).

PRAM (Parallel Random Access Machine) is a theoretical model of a parallel computer consisting of multiple processors that operate synchronously and share a common global memory, where each processor can access any memory location in constant time.

PRAM architecture: A PRAM system contains:

- P processors: $P_1, P_2, \ldots, P_P$
- A shared global memory
- A global clock

Each processor executes its own instructions and can read/write shared memory.

Key assumptions of PRAM model: PRAM assumes:

Synchronous execution: All processors work in steps called parallel time steps. At each time step, all processors execute one instruction.

Uniform memory access time: Just like RAM model, accessing any memory location takes O(1) time.

Shared memory: All processors can access the same memory.

The main issue in PRAM model is: What happens if multiple processors access the same memory location at the same time?

There are two cases:
1. Concurrent read: Many processors read the same location simultaneously.
2. Concurrent write: Many processors write to the same location simultaneously.

These conflicts define different PRAM variants.

Measuring time in PRAM model:

In PRAM, time is called Parallel Time T_p.

In parallel computing, parallel time is the time taken to complete a computation using multiple processors simultaneously.

Basic parallel time formula:

If the work is perfectly divided among processors, then $T_p = \frac{T_1}{p}$
where, T_1= sequential execution time i.e. time using a single processor, p= number of processors and T_p= parallel execution time

Goal in PRAM algorithms:

- small parallel time T_p
- reasonable number of processors p
- low cost

Advantages of PRAM model:

- Simple to understand
- Helps design parallel algorithms easily
- Provides theoretical performance limits
- Useful in complexity analysis

Limitations of PRAM model:

Unrealistic assumptions in PRAM model are:

- infinite bandwidth
- constant time memory access for all processors
- no communication delay

PRAM Variants (EREW, CREW, CRCW):

Why PRAM variants exist?

In PRAM, many processors share the same memory. So, conflicts happen when multiple processors try to access the same memory location at the same time. There are two possible conflicts.
Concurrent read conflict: Many processors read the same location simultaneously.
Concurrent write conflict: Many processors write to the same location simultaneously.
Different PRAM models define different rules for these situations.

Exclusive Read Exclusive Write (EREW) PRAM: EREW PRAM is a PRAM model where no two processors can read from the same memory location at the same time, and no two processors can write to the same memory location at the same time. However, different processors can read different locations simultaneously and also, different processors can write different locations simultaneously.

Advantage:

- Most realistic and strict model
- Avoids conflicts completely

Disadvantage:

- Harder to design fast algorithms
- Sometimes slower than other models

Concurrent Read Exclusive Write (CREW) PRAM: CREW PRAM allows multiple processors to read the same memory location simultaneously, but does not allow multiple processors to write to the same memory location at the same time.

Advantage

- Faster algorithms than EREW
- Easy to broadcast common data

Disadvantage

- Write conflicts still must be avoided

Concurrent Read Concurrent Write (CRCW) PRAM: CRCW PRAM allows multiple processors to read and write the same memory location simultaneously.

If many processors write to the same location at the same time, what value will be stored?

This is handled using CRCW sub-models.

CRCW sub-models:

Common CRCW: All processors must write the same value. If they write different values, this is not allowed / undefined.

Arbitrary CRCW: If multiple processors write different values, one of them is chosen arbitrarily.

Priority CRCW: If multiple processors write, the processor with highest priority (usually smallest processor ID) wins.

Combining CRCW: Writes are combined using an operation like sum, max, min, OR, AND. This is powerful for reductions.

That's why CRCW model is least realistic, but has highest speed potential.

Parallel algorithm design for searching:

Sequential searching (RAM model):
Sequential linear search: Check elements one by one. Worst case time O(n).

Why parallel searching?

If n is very large, sequential search becomes too slow.

Parallel idea: Divide the array among many processors and search simultaneously.

Parallel linear search (PRAM model):

Let, array size = n, number of processors = p

Each processor searches a portion of the array.

Steps:

Split array into p chunks

Processor P_i searches chunk i

If any processor finds the key, then, it produces local_found = 1, otherwise, local_found = 0.

Show global_found using reduction operation

Algorithm for parallel linear search:

```
PARALLEL_LINEAR_SEARCH_REDUCTION(A, n, key, p)

  // Step 1: initialize
  chunk_size ← n / p

  // Each processor keeps local result
  parallel for each processor i = 0 to p-1 do

    local_index ← ∞   // means "not found"

    start ← i * chunk_size
    end ← (i == p-1) ? (n-1) : (start + chunk_size - 1)

    for j ← start to end do
      if A[j] == key then
        local_index ← j
        break

    // Step 2: Reduction (MIN operation)
    global_index ← REDUCE_MIN(local_index)

  // Step 3: Final result
  if global_index != ∞ then
    return TRUE, global_index
```

```
else
    return FALSE, -1
```

Parallel algorithm design for sorting:

Why parallel sorting is harder than parallel searching?

In parallel searching, each processor works independently. So, minimal interaction between processors.

In parallel sorting, elements must move to correct positions. It requires comparison and exchange across processors. Also, needs a lot of coordination among processors. So sorting requires communication, synchronization and data movement.

Parallel bitonic sort:

A sequence is called bitonic if it first increases, then decreases and vice-versa. Example of bitonic sequence: 2, 5, 8, 10, 9, 6, 3

Key idea of bitonic sort:
Bitonic sort works in two phases.

1. Create bitonic sequences: Divide data and sort.

- Sort first half in ascending order
- Sort second half in descending order

2. Bitonic merge: Convert the bitonic sequence into a fully sorted sequence. This is done using compare-and-swap operations in parallel.

Example:

Input: [10, 3, 7, 4, 8, 2, 6, 1]

1. Create bitonic sequences:
Divide into 2 halves. Left half: sort in ascending order, Right half: sort in descending order

Left: [10, 3, 7, 4]

Sort ascending using bitonic method:

- Compare (10,3) → Swap → (3,10)
- Compare (7,4) → Swap → (4,7)

Now: [3, 10, 4, 7]
Merge ascending:

- Compare (3,4) → No change
- Compare (10,7) → Swap

Final left half: [3, 4, 7, 10]

Right: [8, 2, 6, 1]

Sort descending using bitonic method:

- Compare (8,2) → No change
- Compare (6,1) → No change

Now: [8, 2, 6, 1]

Merge descending:

- Compare (8,6) → No change
- Compare (2,1) → No change

Final right half: [8, 6, 2, 1]

Now, full array becomes [3, 4, 7, 10, 8, 6, 2, 1], which is bitonic.

2. Bitonic merge (Parallel): Now, convert this into sorted order.

Compare distance = 4

Compare pairs:

- (3,8) → No change
- (4,6) → No change
- (7,2) → Swap
- (10,1) → Swap

Result: [3, 4, 2, 1, 8, 6, 7, 10]

Compare distance = 2

Left half:

- (3,2) → Swap
- (4,1) → Swap

Right half:

- (8,7) → Swap
- (6,10) → No change

Result: [2, 1, 3, 4, 7, 6, 8, 10]

Compare distance = 1

- (2,1) → Swap
- (3,4) → No change

- (7,6) → Swap
- (8,10) → No change

Final sorted array: [1, 2, 3, 4, 6, 7, 8, 10]

Algorithm (Non-recursive version):

```
PARALLEL_BITONIC_SORT(A, n):

   for k = 2 to n step *= 2:            //This stage builds bitonic sequences of increasing size

      for j = k/2 down to 1 step /= 2:  //Pass inside stage, controls distance between elements being
                                          compared

         parallel for i = 0 to n-1:        //Parallelism happens

           l = i XOR j                      //Finds the element to compare with

           if l > i:

              if ( (i AND k) == 0 AND A[i] > A[l] ) //Decides sorting direction and compares

                 swap A[i], A[l]

              if ( (i AND k) != 0 AND A[i] < A[l] )

                 swap A[i], A[l]
```

Algorithm (Recursive version):

```
BITONIC_SORT(A, low, count, dir):
   if count > 1:
      k = count / 2

      parallel:
         BITONIC_SORT(A, low, k, ASCENDING)
         BITONIC_SORT(A, low + k, k, DESCENDING)
      BITONIC_MERGE(A, low, count, dir)
BITONIC_MERGE(A, low, count, dir):

   if count > 1:
      k = count / 2

      parallel for i = low to low + k - 1:
```

```
        if dir == ASCENDING:
            if A[i] > A[i+k]:
                swap A[i], A[i+k]

        else:
            if A[i] < A[i+k]:
                swap A[i], A[i+k]

    BITONIC_MERGE(A, low, k, dir)
    BITONIC_MERGE(A, low + k, k, dir)
```

Parallel matrix addition:

Algorithm:

```
PARALLEL_MATRIX_ADDITION(A, B, C, n, m)

  // A, B → input matrices
  // C → result matrix
  // n × m size

  parallel for i ← 0 to n-1 do
    parallel for j ← 0 to m-1 do

      // Each (i,j) handled by one processor
      C[i][j] ← A[i][j] + B[i][j]

  return C
```

Parallel matrix multiplication:

Algorithm (One processor per element or c[i][j]):

```
PARALLEL_MATRIX_MULTIPLICATION_ELEMENT(A, B, C, n)

  parallel for i ← 0 to n-1 do
    parallel for j ← 0 to n-1 do

      sum ← 0   // local variable (private)

      for k ← 0 to n-1 do
        sum ← sum + A[i][k] * B[k][j]

      C[i][j] ← sum

  return C
```

Algorithm (One processor per multiplication term):

```
PARALLEL_MATRIX_MULTIPLICATION_TERM(A, B, C, n)

  for i ← 0 to n-1 do

    for j ← 0 to n-1 do

      // Parallel computation over k

      parallel for k ← 0 to n-1 do

        local_value[k] ← A[i][k] * B[k][j]

      // Reduction (sum)

      C[i][j] ← REDUCE_SUM(local_value[0 ... n-1])

  return C
```

Parallel running time, T_p: It is the time taken by a parallel algorithm to solve a problem using p processors. If $p = 1$, then $T_1 =$ sequential time.

Speedup, S_p: Speedup tells how much faster the parallel algorithm is compared to sequential execution. Speedup, $S_p = \frac{T_1}{T_p}$

Types of speedup:

Linear speedup: $S_p = p$, perfect speedup (ideal case)
Sublinear speedup: $S_p < p$, most common (due to overhead)
Superlinear speedup: $S_p > p$, rare

Efficiency, E_p: Efficiency tells how effectively processors are being utilized. Efficiency, $E_p = \frac{S_p}{p}$
Efficiency range: $0 < E_p \leq 1$

Cost: The cost of a parallel algorithm is the total amount of computational work performed by all processors together. Cost=p*T_p , where, p is number of processors, T_p= parallel running time

Why cost is important?

Sometimes a parallel algorithm is fast but may waste resources.
Suppose, 100 processors are used. Each processor does very little useful work. Then, total cost becomes very high. So, cost is needed to judge overall efficiency, resource usage, and whether parallelisation is worthwhile.

Cost-optimal parallel algorithm: A parallel algorithm is cost-optimal if its cost is of the same order as the best sequential time. That is, $pT_p = O(T_1)$, where: $T_1$is the best sequential time.
In simple way, if sequential algorithm needs 100 operations and parallel algorithm also uses about 100 total operations across all processors, then the parallel algorithm is cost-optimal.

When parallel algorithms become not cost-optimal?
If we use too many processors such that, $pT_p > T_1$

Then, total work is more than sequential work, too much overhead and poor efficiency.

Example:
Using $p = n^3$processors for matrix multiplication, $T_p = O(\log\ n)$
Cost, $pT_p = n^3 \log\ n$, This is greater than n^3.
So, not cost-optimal.

Summary of parallel performance metrics for different algorithms:

Algorithm	**Version**	**Parallel Time (Tp)**	**Speedup (S)**	**Efficiency (E)**	**Cost (p·Tp)**	**Cost-Optimal**
Linear search	Reduction-based	O(n/p + log p)	n / (n/p + log p)	1 / (1 + (p log p)/n)	O(n + p log p)	Yes (if p log p = O(n))
Bitonic sort	Standard parallel	O(log^2 n)	O(n)	O(1)	O(n log^2 n)	Yes
Matrix addition	One processor per element	O(1)	O(n^2)	1	O(n^2)	Yes
Matrix multiplication	One processor per element	O(n)	O(n^2)	1	O(n^3)	Yes
Matrix multiplication	One processor per term (Reduction)	O(log n)	O(n^3 / log n)	O(1 / log n)	O(n^3 log n)	No